

This exam consists of 17 questions.

Points are indicated for each question and broken down to each part.

Question 1: Defining Software Engineering (4 points)

Define the term *Software Engineering* as discussed in the lecture. State whether software engineering is considered a *true engineering discipline*. Provide one argument supporting this view and one argument against it.

Question 2: The Software Crisis (6 points)

List and briefly describe two primary reasons that contributed to the *Software Crisis* and briefly describe two symptoms identified in the lecture. State and justify whether software lifecycle models address the reasons or the symptoms.

Question 3: Waterfall vs. Spiral Model (4 points)

Describe the core philosophies of the Waterfall model and the Spiral model. For each lifecycle model, describe and justify the type of software project for which it is most suitable.

Question 4: Prototyping Models (6 points)

Compare *Evolutionary Prototyping* and *Throwaway Prototyping* (one commonality, one difference in the approaches). For each individual approach, state one differentiating advantage and one differentiating disadvantage.

Question 5: Importance of Requirements Engineering (3 points)

Explain the relation between *Software Quality* and *fitness for purpose*. Explain how this makes Requirements Engineering a critical activity for the success of a software development project.

Question 6: Stakeholders and Conflicting Goals (6 points)

For a new mobile banking application, identify two distinct *internal stakeholders* and two distinct *external stakeholders*. For each of the four stakeholders, state one relevant concern they might have. Finally, provide one example of a potential *conflicting goal* between any two of the stakeholders you identified.

Question 7: Functional vs. Non-Functional Requirements (6 points)

Clearly distinguish between *Functional* and *Non-Functional* requirements. Provide two examples of verifiable non-functional requirements for an online video streaming service. For each example, explain why it is non-functional and what makes it verifiable.

Question 8: Ambiguity in Natural Language Requirements (6 points)

Define *ambiguity* in the context of software requirements. Provide a specific example of an ambiguous requirement for an online forum system. Finally, explain a method from the lecture and how it may mitigate ambiguity in your example requirement.

Question 9: Use Case Diagram Relationships (4 points)

Explain the meaning and purpose of the `<<include>>` and `<<extend>>` relationships in UML Use Case Diagrams. For each relationship, provide a clear and distinct example use (excerpt of a Use Case Diagram) in the context of a food delivery application.

Question 10: Model Criteria (6 points)

Define what a *model* is in the context of software engineering based on the three core criteria a representation must satisfy to be considered a model, as discussed in the lecture. Illustrate each criterion on a single software model.

Question 11: Elements of a Modeling Language (6 points)

List and briefly describe the three fundamental elements that define a modeling language. Illustrate these three elements using the example of UML Class Diagrams.

Question 12: Language Selection Criteria (4 points)

List two distinct criteria for selecting a programming language for a software project as discussed in the lecture. For each criterion, provide a brief explanation of its importance.

Question 13: Programming Style and Readability (4 points)

A program is read much more often than it is written. Provide two specific examples of naming conventions discussed in the lecture (e.g., for classes, methods, or constants) and explain how each one improves code readability.

Question 14: Fundamental Testing Concepts (6 points)

Briefly define the terms *Error*, *Failure*, and *Fault* according to the IEEE terminology discussed in the lecture. Provide a single, clear example from a web application (e.g., an e-commerce site) that illustrates the relationship between all three concepts.

Question 15: Practical White-Box Testing and Mutation Testing

Consider the following Java code snippet:

```
1 public int calculate(int a, int b) {  
2     int r = 0;  
3     if (a > 5 && b == 0) {  
4         r = a;  
5     } else {  
6         r = a + b;  
7     }  
8     if (r < 0) {  
9         r = 0;  
10    }  
11    return r;  
12 }
```

(a) (3 points) Provide a minimal set of test cases to achieve maximal branch coverage.

(b) (3 points) Briefly explain what a *live mutant* is in Mutation Testing. Propose one simple mutation to the original code snippet that your test suite from part (a) does **not** kill, and explain why.

Question 16: Extreme Programming (XP) Practices (6 points)

Briefly explain three distinct practices of Extreme Programming (XP) from the lecture. For each chosen practice, describe its main goal.

Question 17: Anti-Patterns vs. Code Smells (5 points)

Briefly explain what an "Anti-Pattern" is and how it differs from a "Code Smell." Provide one example of an anti-pattern from the lecture and explain the common problem it tries to solve and the negative consequences it introduces.

(This is an additional sheet for your answers.)

(This is an additional sheet for your answers.)